

Höhere Informatik

Prof. Probst

Software-Architektur einer Physik-Simulation in C++

Konrad Wehkamp () und Lorenz Saalman (5391121)

SS 26

1 Zusammenfassung

2 Einleitung

Simulationen können ein sehr nützliches Werkzeug sein, um komplexe Systeme zu untersuchen und vorherzusagen. Besonders in der Physik und den Ingenieurwissenschaften sind sie ein fester Bestandteil der Forschung und Entwicklung. Sie werden vor allem dann eingesetzt, wenn es schwierig oder unmöglich ist eine analytische Lösung zu finden, oder wenn es unrealistisch ist ein Experiment durchzuführen. Außerdem ermöglichen sie das Durchführen von Designstudien, da sie schnelle Iterationen und Tests von Konzepten möglich machen. Dieser Bericht beschäftigt sich mit der Software-Architektur einer Physik-Simulation in C++, die die grundlegende Dynamik von Objekten simuliert. Die Simulation löst, beschränkt auf zwei Dimensionen, die Bewegungsgleichungen für eine Vielzahl von Objekten zeitdiskret. Mittels verschiedener Integrationsverfahren kann die Position x_0 und Geschwindigkeit v_0 eines Objekts genutzt werden, um die Position x_1 und Geschwindigkeit v_1 zu einem späteren Zeitpunkt zu approximieren. Das simple Euler-Verfahren berechnet die Position beispielsweise wie folgt:

$$x_1 = x_0 + v_0 * \Delta t \quad (1)$$

$$v_1 = v_0 + a_0 * \Delta t. \quad (2)$$

Dabei ist a_0 die Beschleunigung zum Zeitpunkt t_0 und Δt die diskrete Zeitschrittgröße. Die Beschleunigung eines Objekts wird durch die Summe aller Kräfte bestimmt und kann beispielsweise durch Gravitation oder Kollision mit anderen Objekten verursacht werden.

Die Simulation bildet die Grundlage, um das Verhalten und die Steuerung einer Rakete mit Schubvektorkontrolle zu simulieren. Diese soll als Objekt in der Simulation normal teilnehmen, wobei die Schubkraft als zusätzliche Kraft wirkt.

Diese Ausarbeitung beschreibt, wie die Simulation objekt-orientiert umgesetzt wurde und konzentriert sich dabei auf die höhere Struktur und Architektur der Anwendung. Zunächst werden die Ziele der Simulation konkret definiert, bevor die Umsetzung der Software-Architektur detailliert beschrieben wird. Abschließend werden die Ergebnisse und Erkenntnisse aus der Entwicklung der Simulation diskutiert.

3 Zielsetzung

Die Zielsetzung der Simulation ist es, eine flexible und erweiterbare Software-Architektur zu entwickeln, die es ermöglicht, verschiedene physikalische Szenarien zu simulieren. Die Position der Objekte soll in einem zweidimensionalen Raum grafisch visualisiert werden. Folgende Anforderungen wurden definiert:

- Bewegung von Objekten nach newtonischen Bewegungsgleichungen
- Kollisionserkennung und Lösung mit kreisförmigen Objekten
- Krafteinwirkung auf ein Objekt an beliebiger Position
- globale und von massebehaftenden Objekten ausgehende Gravitationskraft
- Einstellung der Simulation durch den Nutzer (z.B. Zeitschrittgröße, Anzahl und Position der Objekte, etc.)

- Implementierung verschiedener Integrationsverfahren

Zusätzlich soll die autonome Steuerung einer Rakete mit Schubvektorkontrolle simuliert werden. Hierfür wurden folgende Ziele definiert:

- Implementierung einer Rakete mit Schubvektorkontrolle als Objekt in der Simulation
- Implementierung eines Regelalgorithmus, der die Schubvektorkontrolle der Rakete ermöglicht
- Definition von Zielpunkten für den Regelalgorithmus

4 Umsetzung

Die Implementierung erfolgte in C++ unter Verwendung von objekt-orientierten Prinzipien. Der gesamte Code wurde eigenständig geschrieben, welche Bibliotheken verwendet wurden, ist in Tabelle 1 im Anhang vermerkt. Die Umsetzung der grafischen Oberfläche erfolgt mit GLFW und OPENGGL, wobei die Benutzeroberfläche IMGUI verwendet. Der Code, sowie die gesamte Entwicklungshistorie können im GitHub Repository¹ eingesehen werden.

4.1 Herangehensweise

Um möglichst wenig fehleranfälligen und gut erweiterbaren Code zu schreiben, wurden verschiedene Ansätze verfolgt. Grundsätzlich wurde das Projekt in möglichst unabhängige Module aufgeteilt, wie z.B. die Physik-Engine, die grafische Darstellung, die Benutzeroberfläche, etc. Zwischen den Modulen wurden möglichst kompakte Schnittstellen definiert. Einer der wichtigsten Aspekte war die Trennung von Daten, Logik und Darstellung. Wo immer möglich, wurden Abhängigkeiten so gesetzt, dass die übergeordnete Logik keine Annahmen über die Details der Implementierung der Module treffen muss. So konnte vermehrt Gebrauch vom Abhängigkeits-Inversions-Prinzip und dem Strategie-Muster gemacht werden.

4.2 Struktur

Die grundlegende Struktur der Klassen im Projekt ist in Abbildung 1 dargestellt. Der namespace Core beinhaltet die Klasse Application, welche eine simple Hauptschleife ausführt und für einen vector von der Klasse AppLayer deren Ereignisfunktionen OnInit, OnUpdate und OnRender aufruft. Außerdem verwaltet sie ein Ereignis-Beobachter-System, mit dem die AppLayer Ereignisse abonnieren und auslösen kann. Der Code in dem namespace Core ist auf einem OpenSource Git-Repository² basiert und wurde für diese Anwendung angepasst. Der Rest der Anwendung befindet sich im namespace App und besteht aus vier Klassen die AppLayer erweitern. Die Klasse EngineLayer bildet die Basis und verwaltet die Physiks simulation, das Rendering und die Daten. Die Klasse UILayer stellt die Benutzeroberfläche dar, mit der die Simulation gesteuert werden kann. Zusätzlich gibt es den AudioLayer, welcher die Wiedergabe von Audio ermöglicht, und den ControlLayer, welcher die Steuerung einer Rakete in der Simulation übernimmt. Der EngineLayer hängt dabei überhaupt nicht von den anderen AppLayern ab.

¹<https://github.com/grav-byte/GravSim>

²<https://github.com/TheCherno/Architecture>

Die Daten der Simulation werden in der Klasse `Scene` verwaltet, welche einen `vector` von `SceneObject` enthält. Diese Objekte präsentieren physische Objekte in der Simulation und speichern eine Struktur `Transform`, die Position, Rotation und Größe festlegt, sowie Felder für Geschwindigkeit, Masse und andere vom Nutzer definierte Eigenschaften. Außerdem hat jedes Objekt ein `IVisual`, um Daten zur grafischen Darstellung zu speichern und eine Liste von `ColliderBase` für die Kollisionslösung.

Die Struktur verwendet ein modulares MVC-Konzept, in dem die Model-, View- und Controller-Funktionen auf mehrere Klassen verteilt sind, die gemeinsam die jeweilige Schicht bilden. Das Model besteht aus der Klasse `EngineLayer` und den im vorherigen Abschnitt genannten Datenklassen. Der View wird durch alle Klassen des Rendering Moduls gebildet, welche nicht im Detail in dieser Ausarbeitung beschrieben werden, aber die grafische Darstellung der Simulation über OpenGL Shader umsetzen. Der Controller besteht aus der `UILayer`, welche die Benutzeroberfläche darstellt, und der `ControlLayer`, welche die Steuerung einer Rakete ermöglicht. Abhängigkeiten zwischen den Ebenen sind dabei so umgesetzt, dass `EngineLayer` unabhängig von den anderen Ebenen ist, während `UILayer` und `ControlLayer` von `EngineLayer` abhängen, um die Simulation zu steuern.

Tabelle 1: Verwendete Bibliotheken und deren Zweck

Bibliothek	Zweck
cereal	Serialisierung von C++-Objekten zur Speicherung
glfw	Erstellung von Fenstern sowie Verwaltung von Eingaben
imgui	UI-Bibliothek zur Erstellung der grafischen Benutzeroberfläche
miniaudio	Plattformübergreifende Bibliothek zur Wiedergabe von Audio
glew	OpenGL-Erweiterung
glm	Mathematikbibliothek für Vektoren, etc.
implot	Erweiterung von ImGui zur Darstellung von Plots
stb	Header-only-Bibliothek zum Laden von Texturen

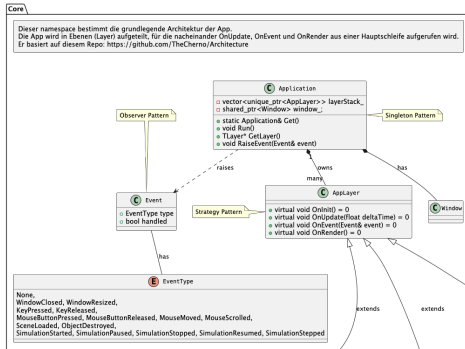


Abbildung 1: Überblick über die oberste Struktur der Anwendung